串

阅读顺序

点标题跳到对应页

串的抽象数据类型 第 3 页

理解空串、子串与模式匹配的返回值约定。

串的存储 第 31 页

对照定长顺序、堆分配和块链三种表示。

朴素匹配 第 45 页

在失配后重置主串的匹配起点。

KMP 匹配 第 49 页

利用已经匹配的部分移动模式串，主串位置不回退。

next 与 nextval 第 58 页

先求失配回退的位置，再看怎样跳过必然失配的比较。

易错点

按本课件的一基下标约定，第 0 项存串长，next[1] 为 0。手算和代码要统一这套约定。

第 62、64 页注意判等与赋值的区别。第 64 页的优化结果写入 nextval。

随堂自测

按课件约定，模式串 abab 的 next 序列是什么？

答案 0、1、1、2。

第四章 串

4.1 串的抽象数据类型的定义

4.2 串的表示和实现

4.3 串的模式匹配算法



4.1 串的抽象数据类型的定义如下：

串是有限长的字符序列，由一对单引号相括，如：'a string'

ADT String {

数据对象：

$$D = \{ a_i \mid a_i \in CharacterSet, \quad i=1,2,...,n, \quad n \geq 0 \}$$

数据关系：

$$R_1 = \{ < a_{i-1}, a_i > \mid a_{i-1}, a_i \in D, \quad i=2,...,n \}$$

基本操作：

StrAssign (&T, chars)

DestroyString(&S)

StrCopy (&T, S)

StrLength(S)

StrCompare (S, T)

Concat (&T, S1, S2)

StrEmpty (S)

SubString (**&**Sub, S, pos, len)

ClearString (**&**S)

Index (S, T, pos)

Replace (**&**S, T, V)

StrInsert (**&**S, pos, T)

StrDelete (**&**S, pos, len)

} ADT String



StrAssign (&T, chars)

初始条件：chars 是字符串常量。

操作结果：把 chars 赋为 T 的值。

StrCopy (&T, S)

初始条件：串 S 存在。

操作结果：由串 S 复制得串 T。

DestroyString (&S)

初始条件：串 S 存在。

操作结果：串 S 被销毁。

StrEmpty(S)

初始条件： 串S存在。

操作结果： 若 S 为空串，则返回TRUE，
否则返回 FALSE。

'' 表示空串，空串的长度为零。

StrCompare(S,T)

初始条件：串 S 和 T 存在。

操作结果：若S > T，则返回值 > 0；

若S = T，则返回值 = 0；

若S < T，则返回值 < 0。

例如：StrCompare('data', 'state') < 0

StrCompare('cat', 'case') > 0

StrLength(S)

初始条件：串 S 存在。

操作结果：返回 S 的元素个，

称为串的长度。

Concat(&T, S1, S2)

初始条件：串 S1 和 S2 存在。

操作结果：用 T 返回由 S1 和 S2 联接而成的新串。

例如： Concate(T, 'man', 'kind')

求得 T = 'mankind'

SubString (&Sub, S, pos, len)

初始条件:

串 S 存在，**1≤pos≤StrLength(S)**

且**0≤len≤StrLength(S)-pos+1**。

操作结果:

用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。

子串为“串”中的一个字符子序列

例如：

SubString(sub, 'commander', 4, 3)

求得 sub = 'man' ;

SubString(sub, 'commander', 1, 9)

求得 sub = 'commander';

SubString(sub, 'commander', 9, 1)

求得 sub = 'r';

SubString(sub, 'commander', 4, 7)

sub = ?

SubString(sub, 'beijing', 7, 2) = ?

sub = ?

- **起始位置和子串长度之间存在约束关系**

SubString('student', 5, 0) = ''

- **长度为 0 的子串为“合法”串**

Index(S,T,pos)

初始条件：串S和T存在，T是非空串，

$1 \le pos \le StrLength(S)$ 。

操作结果：若主串 S 中存在和串 T 值相同的子串，则返回它在主串 S 中第pos个字符之后第一次出现的位置；否则函数值为0。

“**子串在主串中的位置**”意指子串中的第一个字符在主串中的**位序**。

假设 S = 'abcaabcaaabc', T = 'bca'

Index(S, T, **1**) = 2;

Index(S, T, **3**) = 6;

Index(S, T, **8**) = 0;

Replace (&S, T, V)

初始条件： 串S，T和 V 均已存，且 T 是非空串。

操作结果： 用V替换主串S中出现的所有与（模式串）T 相等的不重叠的子串。

例如：

假设 S = 'abcaabcaaabca'，T = 'bca'

若 V = 'x'，则经置换后得到

S = 'axaxaax'

若 V = 'bc'，则经置换后得到

S = 'abcabcaabc'



StrInsert (&S, pos, T)

初始条件：串S和T存在，

$1 \le pos \le StrLength(S) + 1$。

操作结果：在串S的第pos个字符之前插入串T。

例如： S = 'chater'，T = 'rac'，

则执行 StrInsert(S, 4, T)之后得到

S = 'character'



StrDelete (&S, pos, len)

初始条件：串S存在

$1 \le pos \le StrLength(S)-len+1$。

操作结果：从串S中删除第pos个字符起长度为len的子串。

ClearString(&S)

初始条件：串S存在。

操作结果：将S清为空串。



对于串的基本操作集可以有不同的定义方法，在使用高级程序设计语言中的串类型时，应**以该语言的参考手册为准。**

例如：C语言函数库中提供下列串处理函数：

gets(str) 输入一个串；

puts(str) 输出一个串；

strcat(str1, str2) 串联接函数；

strcpy(str1, str2, k) 串复制函数；

strcmp(str1, str2) 串比较函数；

strlen(str) 求串长函数；

在上述抽象数据类型定义的13种操作中，

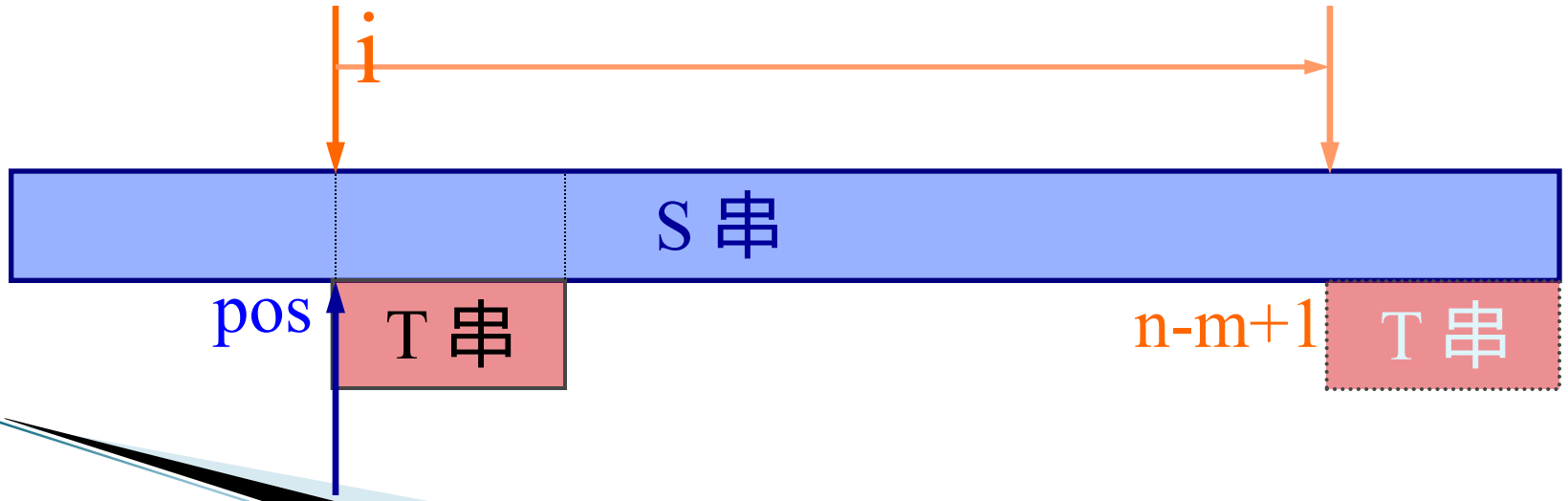
串赋值StrAssign、串复制Strcopy、串比较StrCompare、求串长StrLength、串联接Concat以及求子串SubString等六种操作构成串类型的最小操作子集。

即：这些操作不可能利用其他串操作来实现，反之，其他串操作（除串清除ClearString和串销毁DestroyString外）可在这个最小操作子集上实现。

例如，可利用串比较、求串长和求子串等操作实现定位函数Index(S,T,pos)。

算法的基本思想为：

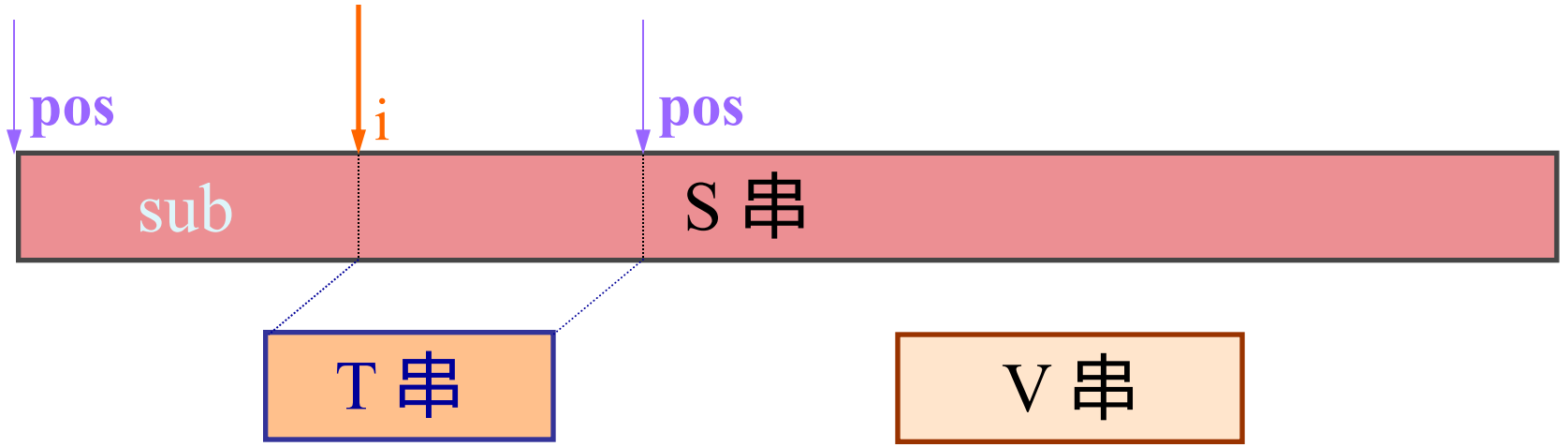
StrCompare(SubString(S, i, StrLength(T)),T) ?=0

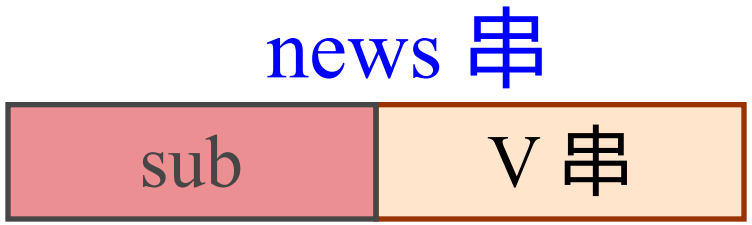


```
int Index (String S, String T, int pos) {
// T为非空串。若主串S中第pos个字符之后存在与 T相等的子串，
则返回第一个  这样的子串在S中的位置，否则返回0

if (pos > 0) {
   n = StrLength(S);  m = StrLength(T);  i = pos;
   while ( i <= n-m+1) { // 末个起点须留足 m 个字符
      SubString (sub, S, i, m);
      if (StrCompare(sub,T) != 0)   ++i ;
      else return i ;
   } // while
  } // if
  return 0;          // S中不存在与T相等的子串
} // Index
```

又如串的置换函数：





```
void Replace (&S,T,V){
pos=Index(S,T,1);
if(pos!=0){
substring(&sub,S,1,pos-1);
substring (&sub1,S,pos+strlength(T),
        strlength(s)-(pos+strlength(T))+1)
concat(&R,sub,V);
concat(&S,R,sub1);
pos=index(S,T,pos+strlength(V));
}
}
```

串的逻辑结构和线性表极为相似，**区别**仅在于串的数据对象约束为字符集。

串的基本操作和线性表有很大差别。

在线性表的基本操作中，**大多以“单个元素”作为操作对象**；

在串的基本操作中，**通常以“串的整体”作为操作对象。**



4.2 串的表示和实现

在程序设计语言中，串只是作为输入或输出的常量出现，则只需存储此串的串值，即字符序列即可。但在多数非数值处理的程序中，串也以变量的形式出现。

一、串的定长顺序存储表示

二、串的堆分配存储表示

三、串的块链存储表示

一、串的定长顺序存储表示

```
#define  MAXSTRLEN  255
        // 用户可在255以内定义最大串长
 typedef unsigned char  Sstring
                              [MAXSTRLEN + 1];
          // 0号单元存放串的长度
```

特点:

- **串**的实际长度可在这个予定义长度的范围内随意设定，超过予定义长度的串值则被舍去，称之为“**截断**”。

- 按这种串的表示方法实现的串的运算时，其基本操作为“**字符序列的复制**”。

串长过小，则串空间浪费较大

例如：串的联接算法中需分三种情况处理：

```
Status Concat(SString S1, SString S2, SString &T) {
// 用T返回由S1和S2联接而成的新串。若未截断, 则返回TRUE，否则FALSE。
if (S1[0]+S2[0] <= MAXSTRLEN) {// 未截断
  T[1..S1[0]] = S1[1..S1[0]];
  T[S1[0]+1..S1[0]+S2[0]] = S2[1..S2[0]];
else if (S1[0]<MAXSTRSIZE) {// 截断
  T[1..S1[0]] = S1[1..S1[0]];
  T[S1[0]+1..MAXSTRLEN] =
    T[0..MAXSTRLEN] = S1[0..MAXSTRLEN];
                T[0] == S1[0] == MAXSTRLEN
    uncut = FALSE;          }
return uncut;
} // Concat
```



二、串的堆分配存储表示

```
typedef struct {
  char *ch;
     // 若是非空串，则按串长分配存储区，
     //  否则ch为NULL
  int  length;   // 串长度
} HString;
```

通常，C语言中提供的串类型就是以这种存储方式实现的。系统利用函数malloc( )和free( )进行串值空间的动态管理，为每一个新产生的串分配一个存储区，称串值共享的存储空间为“**堆**”。

C语言中的串以一个空字符为结束符，串长是一个隐含值。

这类串操作实现的算法为：

先为新生成的串分配一个存储空间，然后进行串值的复制。

```
Status Concat(HString &T, HString S1, HString S2) {
  // 用T返回由S1和S2联接而成的新串
  if (T.ch)  free(T.ch);        // 释放旧空间
  if (!(T.ch = (char *)
          malloc((S1.length+S2.length)*sizeof(char))))
      exit (OVERFLOW);
  T.ch[0..S1.length-1] = S1.ch[0..S1.length-1];
  T.length = S1.length + S2.length;
  T.ch[S1.length..T.length-1] = S2.ch[0..S2.length-1];
  return OK;
} // Concat
```

```
Status SubString(HString &Sub, HString S,
                                        int pos, int len) {
 // 用Sub返回串S的第pos个字符起长度为len的子串
  if (pos < 1  ||  pos > S.length
                ||  len < 0  ||  len > S.length-pos+1)
      return ERROR;
  if (Sub.ch)  free (Sub.ch);            // 释放旧空间
  if (!len)
    { Sub.ch = NULL;  Sub.length = 0; } // 空子串
  else {      …  …                       } // 完整子串
  return OK;
} // SubString
```



```
Sub.ch = (char *)malloc(len*sizeof(char));
Sub.ch[0..len-1] = S[pos-1..pos+len-2];
Sub.length = len;
```



三、串的块链存储表示

也可用链表来存储串值，由于串的数据元素是一个字符，它只有 8 位二进制数，因此用链表存储时，通常一个结点中存放的不是一个字符，而是一个子串。

$$\text{存储密度} = \frac{\text{数据元素所占存储位}}{\text{实际分配的存储位}}$$

```
#define  CHUNKSIZE  80  // 可由用户定义的块大小
 typedef  struct Chunk {  // 结点结构
   char  ch[CUNKSIZE];
   struct Chunk  *next;
 } Chunk;
 typedef struct {  // 串的链表结构
   Chunk *head, *tail; // 串的头和尾指针
    int   curlen;     // 串的当前长度
 } LString;
```

实际应用时，可以根据问题所需来设置结点的大小。

例如: 在编辑系统中，整个文本编辑区可以看成是一个串，每一行是一个子串，构成一个结点。即: 同一行的串用定长结构(80个字符)，行和行之间用指针相联接。



4.3 串的模式匹配算法

这是串的一种重要操作，很多软件，若有**“编辑”**菜单项的话，则其中必有**“查找”**子菜单项。

首先，回忆一下串匹配(查找)的定义:

INDEX (S, T, pos)

初始条件：串S和T存在，T是非空串，
$1 \le pos \le StrLength(S)$。

操作结果：若主串S中存在和串T值相同的子串返回它在主串S中第pos个字符之后第一次出现的位置;否则函数值为0。

模式匹配——简单算法

```
int Index(SString S, SString T, int pos) {
   // 返回子串T在主串S中第pos个字符之后的位置。若不存在，
   // 则函数值为0。其中，T非空，1≤pos≤StrLength(S)。
    i = pos;   j = 1;
    while (i <= S[0] && j <= T[0]) {
      if (S[i] == T[j]) { ++i;  ++j; }   // 继续比较后继字符
      else { i = i-j+2;   j = 1; }      // 指针后退重新开始匹配
    }
   if (j > T[0])  return  i-T[0];
   else return 0;
} // Index
```



简单的模式匹配算法的时间复杂度分析:

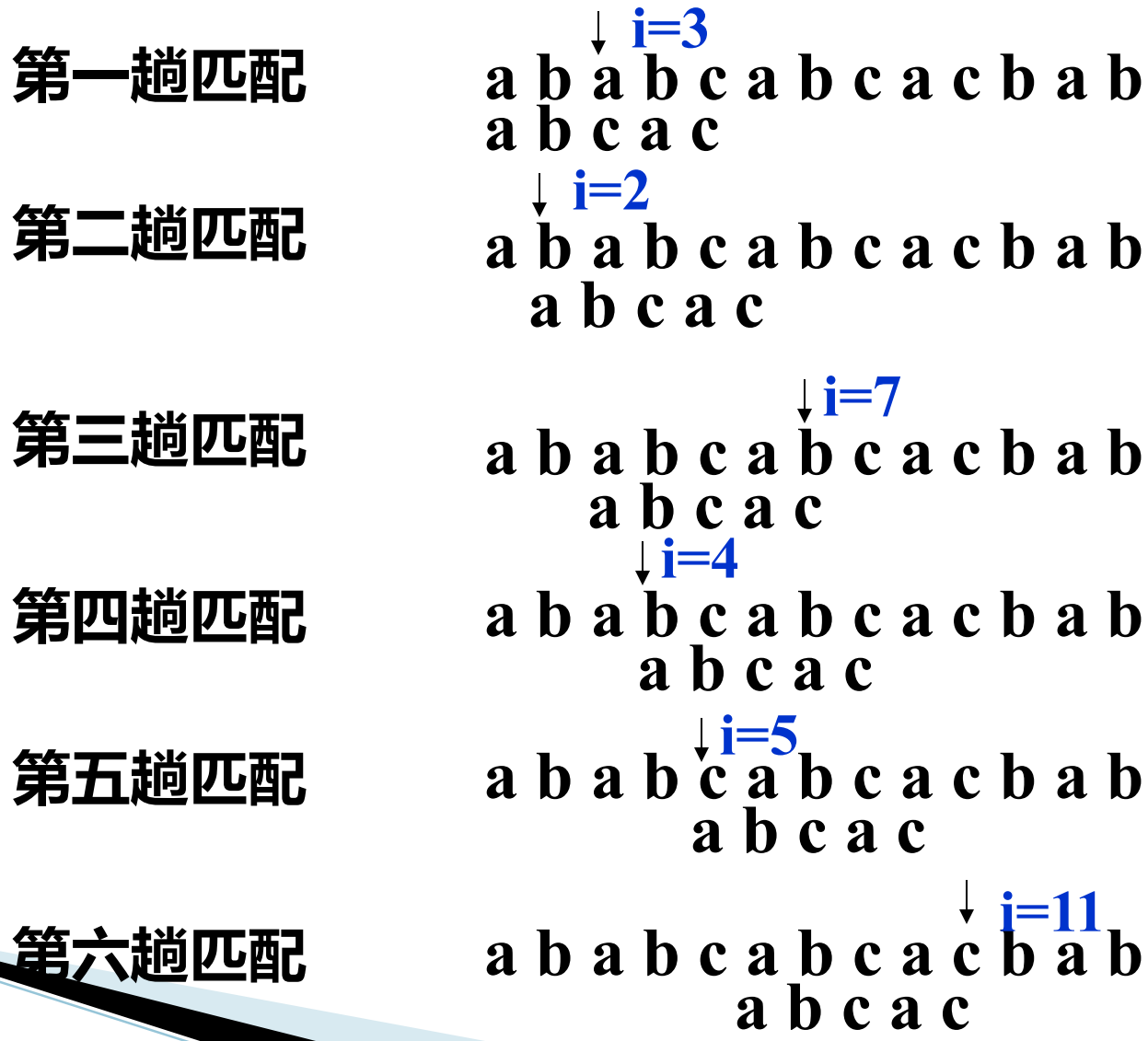
- 该算法的思想简单，易于理解，但算法的效率不高，原因是回溯，分析如下：
  - 最好情况下： O(n+m)
  - 最坏情况下： O(n*m)

改进算法

由D.E.Knuth与J.H.Morris 和V.R.Pratt同时发现的。简称 KMP算法。

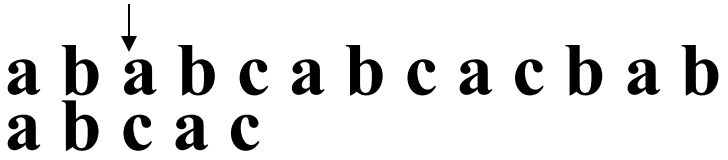
设主串S=“ababcabcacbab”, 模式串T=“abcac”。则朴素算法的匹配过程如下：

未改进时的匹配情况：

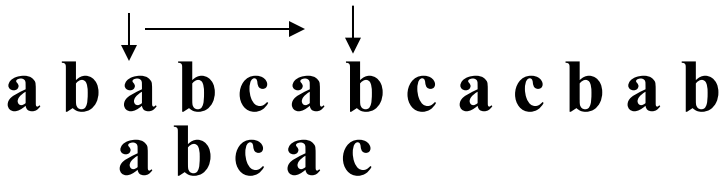

第一趟匹配
i=3
a b a b c a b c a c b a b
a b c a c
第二趟匹配
i=2
a b a b c a b c a c b a b
a b c a c
第三趟匹配
i=7
a b a b c a b c a c b a b
a b c a c
第四趟匹配
i=4
a b a b c a b c a c b a b
a b c a c
第五趟匹配
i=5
a b a b c a b c a c b a b
a b c a c
第六趟匹配
i=11
a b a b c a b c a c b a b
a b c a c

改进后的匹配情况：

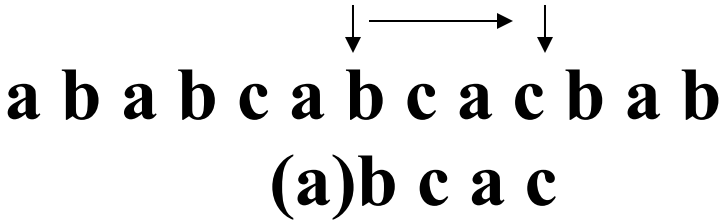
第一趟匹配



第二趟匹配



第三趟匹配



模式匹配——KMP算法



为什么简单算法时间性能低？

在每趟匹配不成功时存在大量回溯，没有利用已经部分匹配的结果。



如何在匹配不成功时主串不回溯？

主串不回溯，模式就需要向右滑动一段距离。



如何确定模式的滑动距离？

模式匹配——KMP算法

结论： i可以不回溯，模式向右滑动到的新比较起点k ，并且k仅与模式串T有关！

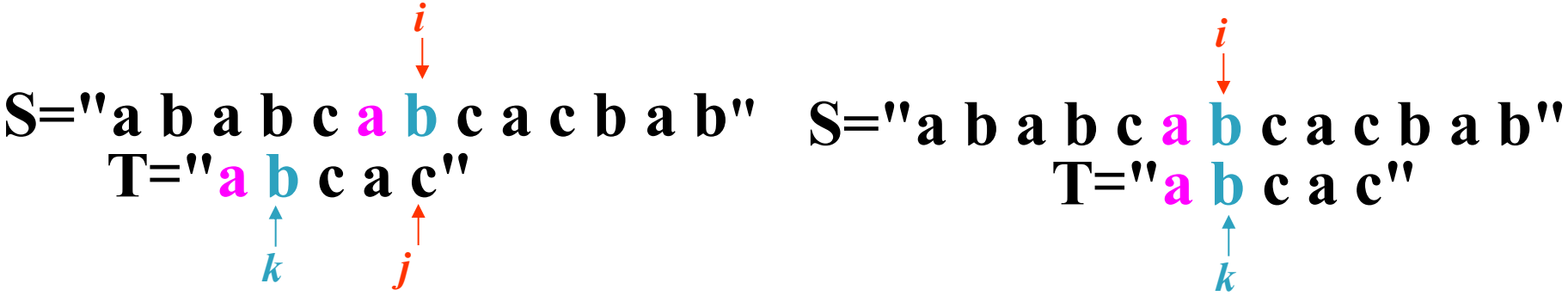
需要讨论两个问题：

①如何由当前部分匹配结果确定模式向右滑动的新比较起点k?

②模式应该向右滑多远才是最高效率的?

模式匹配——KMP算法

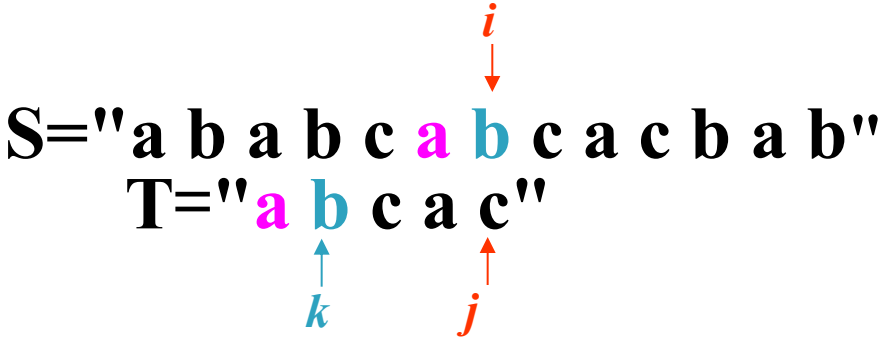
请抓住部分匹配时的两个特征：

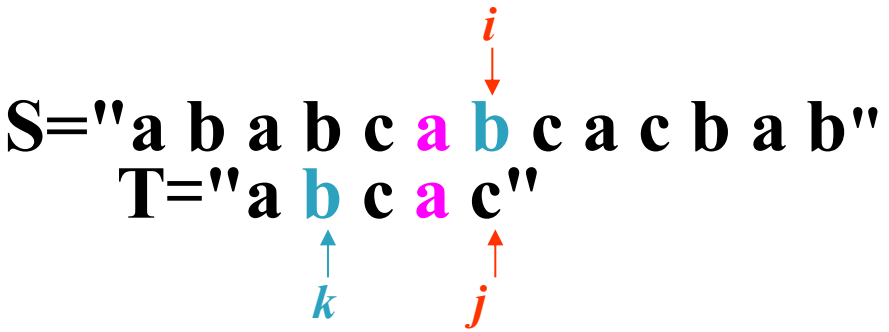


（1）设模式滑动到第k个字符，则$T_1 \sim T_{k-1} = S_{i-(k-1)} \sim S_{i-1}$

模式匹配——KMP算法

请抓住部分匹配时的两个特征：





（1）设模式滑动到第k个字符，则$T_1 \sim T_{k-1} = S_{i-(k-1)} \sim S_{i-1}$

（2）则$T_{j-(k-1)} \sim T_{j-1} = S_{i-(k-1)} \sim S_{i-1}$

两式联立可得： $T_1 \sim T_{k-1} = T_{j-(k-1)} \sim T_{j-1}$

模式匹配——KMP算法

$T_1 \cdots T_{k-1} = T_{j-(k-1)} \cdots T_{j-1}$说明了什么?

（1） k 与 j 具有函数关系，由当前失配位置 j ，可以计算出滑动位置 k（即比较的新起点）；

（2）滑动位置k 仅与模式串T有关。

$T_1 \cdots T_{k-1} = T_{j-(k-1)} \cdots T_{j-1}$的物理意义是什么?

从第1位往右
经过k-1位

从j-1位往左
经过k-1位

模式应该向右滑多远才是最高效率的?

$k = \max \{ k \mid 1<k<j$ 且$T_1 \cdots T_{k-1} = T_{j-(k-1)} \cdots T_{j-1} \}$

模式匹配——KMP算法

- 若令next[j] = k, 则next[j]表明当模式中第j个字符与主串中相应字符“失配”时，在模式串中需重新和主串中该字符进行比较的字符的位置。由此可以得出next函数的定义：

$$next[j]=\begin{cases}0 & \text{当 } j=1 \text{ 时}\\ \text{Max}\{k \mid 1<k<j \text{ 且 } 'p_1p_2\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'\} & \\ 1 & \text{其它情况}\end{cases}$$

模式匹配——KMP算法

模式串 T：	a	b	a	a	b	c	a	c
可能失配位 j：	1	2	3	4	5	6	7	8
新匹配位k=next[j]：	0	1	1	2	2	3	1	2

j=1时, next[j] = 0;

j=2时, next[j] = 1;

j=3时, $T_1 \neq T_2$，因此，k=1;

j=5时, $T_1 = T_4$，因此，k=2;

j=4时, $T_1 = T_3$，因此，k=2;

模式匹配——KMP算法

KMP算法用伪代码描述

1. 在串S和串T中分别设比较的起始下标i和j；
2. 循环直到S中所剩字符长度小于T的长度或T中所有字符均比较完毕

   2.1 如果S[i]=T[j]，继续比较S和T的下一个字符；否则

   2.2 将j向右滑动到next[j]位置，即j=next[j]；

   2.3 如果j=0，则将i和j分别加1，准备下一趟比较；
3. 如果T中所有字符均比较完毕，则返回匹配的起始下标；否则返回0；

模式匹配——KMP算法

```
int Index_KMP(SString S, SString T, int pos) {
    i = pos;   j = 1;
    while (i <= S[0] && j <= T[0]) {
      if (j = = 0 || S[i] = = T[j])
          { ++i;  ++j; }          // 继续比较后继字符
      else  j = next[j];          // 模式串向右移动
    }
    if (j > T[0])  return  i-T[0];          // 匹配成功
    else return 0; // 主串扫描完仍未匹配
}
```



模式串的next数组的生成？

✓ 从前面的讨论可知，next函数值仅取决于模式串本身，而与主串无关。

✓ next[j]的值等于在“$p_1 p_2 \cdots p_{k-1} p_k \ldots p_{j-1}$”这个模式串中，相同的前缀子串和后缀子串的最大长度加1。因此要计算next[j]就要在“$p_1 p_2 \cdots p_{k-1} p_k \ldots p_{j-1}$”找出前缀和后缀相同的最大子串。这个查找过程实际上仍然是模式匹配，只是匹配的模式与目标在这里是同一个串P。

求*next*函数值的过程是一个递推过程，分析如下:

已知：$next[1] = 0;$

假设：$next[j] = k;$ 又 $T[j] = T[k]$

则有：$'p_1 p_2 \cdots p_k' = 'p_{j-k+1} \cdots p_{j-1}'$

即：$next[j+1] = k+1$

若：$T[j] \neq T[k]$

则需往前回朔，检查 $T[j] = T[?]$

假设：next[k] = k’； 又 T[j] = T[k’]

则有：$'p_1p_2\cdots p_{k'-1}'='p_{j-k'+1}\cdots p_{j-1}'$

则：next[j+1] = k+1

若：T[j] ≠ T[k’]

则将模式串继续向右滑动直到将模式中第next[k’]个字符和p_j对齐,…依次类推，直至p_j和模式中某个字符匹配成功或者不存在任何k’满足式(1)，则next[j+1] = 1

这实际上也是一个匹配的过程，

不同在于：主串和模式串是同一个串

例：图中的模式串，已求得前6个字符的next函数值，现求next[7]。

j	1	2	3	4	5	6	7	8
模式	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	1	2

因为next[6]=3,又p6 ≠p3，则需比较p6和p1（因为next[3]=1,）这相当于字串向右滑动

p6 ≠p1，而且next[1]=0,）所以next[7]=1;

而因为p7 =p1，则 next[8]=2。

```
void get_next(SString &T, int &next[] ) {
  // 求模式串T的next函数值并存入数组next
  i = 1;   next[1] = 0;   j = 0;
  while (i < T[0]) {
      if (j = 0 || T[i] == T[j]) // 判等应写 j == 0
          {++i;  ++j; next[i] = j; }
      else  j = next[j]; // 沿较短的前后缀继续尝试
  }
} // get_next
```

还有一种特殊情况需要考虑：

例如：

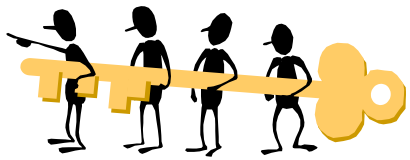
S = 'aaabaaaabaaaabaaaabaaaab'

T = 'aaaab'

next[j]=01234

nextval[j]=00004

```
void get_nextval(SString &T, int &nextval[]) {
  i = 1;   nextval[1] = 0;   j = 0;
  while (i < T[0]) {
    if (j = 0 || T[i] == T[j]) {
      ++i;  ++j;
      if (T[i] != T[j])  next[i] = j; // 此处应写 nextval[i] = j
      else  nextval[i] = nextval[j]; // 相同字符会重复失配，可继续跳过
    }
    else  j = nextval[j];
  }
} // get_nextval
```



本章学习要点

1. 熟悉串的七种基本操作的定义，并能利用这些基本操作来实现串的其它各种操作的方法。

2. 熟练掌握在串的定长顺序存储结构上实现串的各种操作的方法。

3. 了解串的堆存储结构以及在其上实现串操作的基本方法。

4. 理解串匹配的KMP算法，熟悉NEXT函数的定义，学会手工计算给定模式串的NEXT函数值和改进的NEXT函数值。

5. 了解串操作的应用方法和特点。

